

SATQUERY AI

Master Project Guide, Component Breakdown & Phase-Based Implementation Specification

Problem Statement

SIH26167 — SatQuery AI
Indian Space Research Organisation (ISRO)
Theme: Space Technology
Category: Software

1. PROJECT OBJECTIVE

Build a production-oriented interactive web application called **SatQuery AI** that allows a user to upload real remote-sensing imagery and ask questions using natural language.

The system must automatically determine what analysis the user is requesting, select the correct remote-sensing model/toolchain, execute the analysis on the supplied imagery, and return:

- Natural-language answer
- Visual evidence
- Spatial result where applicable
- Confidence information
- Source/image metadata
- Models/tools used
- Processing information
- Downloadable result/report

The system is NOT a chatbot with satellite images attached to it.

It is an **agentic remote-sensing analysis platform**.

The architecture must follow this principle:

AI interprets and plans. Deterministic GIS/raster tools perform scientific calculations, coordinate transformations, geometry processing and measurements.

This separation is explicitly established in the current architecture.

2. ABSOLUTE DEVELOPMENT RULES

These rules apply to every phase and every developer.

RULE 1 — NO SIMULATION

Do NOT create:

- Simulated satellite imagery
- Simulated SAR images
- Simulated optical images
- Simulated change maps
- Simulated model outputs
- Fake confidence values
- Fake coordinates
- Fake areas
- Fake detections
- Fake segmentation masks
- Fake benchmark scores
- Simulated telemetry
- Simulated sensor behaviour
- Artificial “satellite-like” images for demonstrations

The final application must operate on actual imagery and actual model/tool outputs.

A placeholder implementation may exist only during isolated software scaffolding, but it MUST NOT be used as a final feature, benchmark result, or judge-facing demonstration.

3. ZERO SYNTHETIC DATA POLICY

Do not generate synthetic training, validation, testing or demonstration data.

Do not create artificial image pairs such as:

Image A + artificially modified Image A = “change dataset”.

Do not create fake optical/SAR pairs.

Do not invent labels.

Do not generate synthetic captions, synthetic VQA answers, synthetic bounding boxes or synthetic masks to make the system appear functional.

Only use:

- Official public benchmark datasets
- Official open-source remote-sensing datasets
- Real user-provided imagery
- Real imagery obtained through explicitly approved real data sources

Dataset provenance must always be retained.

4. ZERO STATIC DATA POLICY

The application must NOT depend on hardcoded demonstration data.

Do NOT hardcode:

- Example satellite images inside the application
- Fixed answers
- Fixed detections
- Fixed GeoJSON results
- Fixed change percentages
- Fixed area calculations
- Fixed confidence scores
- Fixed benchmark results
- Fixed model responses
- Fixed “expected output” cards presented as live results
- Hardcoded map polygons pretending to be inference results

Every displayed analysis result must originate from the current user request and/or an actual benchmark execution.

Static configuration is permitted where technically necessary, such as:

- Model names
- Environment variables
- Supported file extensions
- Threshold configuration
- API route definitions
- Processing parameters
- Dataset locations
- Feature flags

Static configuration must NEVER be confused with static result data.

5. NO FAKE FALLBACKS

If a model is unavailable, the system must NOT fabricate an answer.

Instead:

1. Detect the failure.
2. Record the failure.
3. Explain which capability is unavailable.
4. Use a legitimate deterministic fallback where scientifically valid.
5. Otherwise return a clearly marked failure/incomplete-analysis result.

For example:

GOOD:

“Grounding model unavailable. Object grounding could not be completed.”

BAD:

“The system could not run Grounding DINO, so this predefined bounding box is displayed.”

The architecture already specifies graceful degradation rather than invented output.

6. CORE FUNCTIONAL REQUIREMENTS

The system must support all mandatory PS capabilities.

Capability A — Single Image VQA

Input:

- One optical/multispectral image OR
- One SAR image
- Natural-language question

Output:

- Answer
- Evidence
- Model used
- Confidence
- Image metadata

Example:

“Describe the land-cover and major objects visible in this image.”

Capability B — Additional Single Image Task

Implement text-guided grounding and segmentation as the preferred additional capability.

Pipeline:

User text → Grounding DINO → candidate bounding boxes → SAM 2.1 → segmentation mask → raster-to-vector → GeoJSON → map display

This follows the existing architecture.

Example:

“Highlight the buildings.”

Capability C — Bi-temporal Analysis

Input:

- Image at T1
- Image at T2
- Same geographic region
- User query

System must:

1. Validate compatibility.
2. Verify CRS/grid/resolution/bounds.
3. Perform registration only when required.
4. Calculate appropriate change metrics.
5. Generate change result.
6. Describe the change.
7. Generate spatial evidence when possible.
8. Return confidence/provenance.

Example:

“What changed between these two dates, and where did the change occur?”

The architecture explicitly defines this flow.

Capability D — Optical + SAR Joint Analysis

Input:

- Optical/multispectral image
- SAR image
- Same geographic area
- User question

System must actually combine information from both modalities.

Example:

“Use the optical and SAR images together to identify built-up and water-covered regions.”

The cross-modal pair requirement is mandatory in the PS.

7. AGENTIC REQUIREMENT

The application must contain an actual controlled agentic workflow.

The controller must:

1. Read the query.
2. Inspect number of images.
3. Inspect modality.
4. Inspect metadata.
5. Determine the requested task.
6. Select permitted tools/models.
7. Execute the workflow.
8. Combine outputs.
9. Construct evidence.
10. Generate final response.
11. Record the execution trace.

The agent **MUST NOT** be an unrestricted autonomous loop.

Use explicit states/nodes and allow-listed tools.

The current architecture already defines this approach using LangGraph.

8. HIGH-LEVEL SYSTEM

```
USER
|
|-- Upload image(s)
|-- Enter natural-language query
|
v
FRONTEND
|
v
FASTAPI API
|
v
INPUT VALIDATION
|
v
AGENT / LANGGRAPH
|
+--> SINGLE IMAGE VQA
|
+--> GROUNDING / SEGMENTATION
|
```

```

+--> BI-TEMPORAL CHANGE ANALYSIS
|
+--> OPTICAL + SAR ANALYSIS
|
v
SPECIALIST MODEL / GIS TOOL
|
v
EVIDENCE ENGINE
|
+--> Measurements
+--> Geometry
+--> Confidence
+--> Provenance
+--> Model outputs
+--> Quality checks
|
v
RESPONSE SYNTHESIS
|
v
WEB UI
|
+--> Text answer
+--> Map
+--> Image layers
+--> Geometry
+--> Metrics
+--> Confidence
+--> Execution trace
+--> Report

```

9. COMPONENT OWNERSHIP

Use a five-person ownership structure.

PERSON 1 — GIS / RASTER / REGISTRATION ENGINEER

Primary ownership:

- GDAL
- Rasterio
- GeoTIFF/COG
- CRS validation
- Affine transforms
- Raster windowing
- AOI extraction
- NDVI

- NDWI
- SAR metrics
- Change masks
- Polygonization
- GeoPandas
- Shapely
- PyProj
- AROSICS

Responsibilities:

Input processing

- Validate raster readability.
- Validate dimensions.
- Validate bands.
- Validate NoData.
- Validate CRS.
- Extract raster metadata.

Analytical branch

Maintain original quantitative raster values.

Never convert analytical raster data to 8-bit before scientific computation.

The architecture explicitly requires analytical values to remain UInt16/Float32 where applicable.

Change analysis

Implement:

- Optical change
- SAR change
- AOI quality calculation
- Cloud/quality masking
- Thresholding
- Noise removal
- Connected components
- Polygonization
- Area calculation

Registration

First perform compatibility checks.

Only invoke AROSICS when registration is actually required.

Do not make AROSICS a hard dependency for already-aligned benchmark data.

10. PERSON 2 — COMPUTER VISION / REMOTE-SENSING AI ENGINEER

Primary ownership:

- Qwen3-VL
- Grounding DINO
- SAM 2.1
- Remote-sensing adaptation
- Inference pipeline
- Model evaluation

Responsibilities:

VQA

Implement real inference on remote-sensing imagery.

Grounding

Text:

“Find buildings.”

→ Grounding DINO

Output:

Bounding boxes + model confidence.

Segmentation

Grounding result → SAM 2.1 → binary mask

Remote-sensing adaptation

At least one visual/VLM component must be adapted using the permitted remote-sensing training data.

This is mandatory because a generic VLM alone does not satisfy the PS.

The adaptation process must have:

- Dataset provenance
- Training configuration
- Training/validation split
- Checkpoints
- Evaluation results

- Version information

Do NOT claim a model is “fine-tuned” unless actual training/adaptation has occurred.

11. PERSON 3 — AGENT / BACKEND ENGINEER

Primary ownership:

- FastAPI
- LangGraph
- Pydantic
- API architecture
- Tool registry
- Evidence schema
- Job state
- Error handling
- Execution trace

The current architecture defines the core workflow state as:

```
query
imagery_metadata
AOI
task_plan
tool_results
evidence
quality
final_response
```

and defines explicit processing nodes.

Required agent nodes

```
validate_input
  ↓
inspect_metadata
  ↓
route_intent
  ↓
prepare_context
  ↓
select_tools
  ↓
execute
  ↓
quality_check
  ↓
```

```
build_evidence
  ↓
synthesize
  ↓
return_result
```

Intent classes

Minimum:

```
scene_description
visual_question_answering
grounding
segmentation
change_detection
change_vqa
optical_sar_analysis
spatial_measurement
multi_step_analysis
```

12. PERSON 4 — FRONTEND / GEO-VISUALIZATION ENGINEER

Primary ownership:

- React
- TailwindCSS
- MapLibre GL or Leaflet
- Image viewer
- Raster layers
- GeoJSON layers
- Temporal comparison
- Query interface
- Results interface
- Execution status

Frontend must contain:

Query panel

User enters natural-language question.

Input panel

Supports:

- Single image
- Two-image temporal pair
- Optical + SAR pair

Processing status

Show actual execution:

```
graph TD; A[Input validation] --> B[Intent detected]; B --> C[Tools selected]; C --> D[Processing]; D --> E[Evidence generated]; E --> F[Answer generated];
```

Do NOT display fake progress animation unrelated to backend state.

The architecture specifically calls for streaming node/tool status.

Map

Display:

- Original image
 - Detection boxes
 - Segmentation masks
 - Change polygons
 - AOI
 - GeoJSON
 - Measurements
-

13. PERSON 5 — DATA / BENCHMARK / EVALUATION / INTEGRATION ENGINEER

Primary ownership:

- Dataset preparation
- Dataset validation
- Benchmark pipelines
- Ground-truth evaluation
- Test cases
- Metrics
- Provenance
- Documentation
- Integration testing
- Demo data management

Must manage only real datasets.

Required public datasets include:

1. BigEarthNet.txt
2. VRSBench
3. RSVQA
4. CDVQA

The PS explicitly assigns these datasets to adaptation and evaluation tasks.

Also prepare the system for the undisclosed ISRO/SAC evaluation dataset.

14. PHASE STRUCTURE

The entire project must be implemented in phases.

Do NOT attempt the full system simultaneously.

PHASE 0 — PROJECT FOUNDATION

Goal:

Create the repository and engineering foundation.

Deliver:

```
frontend/  
backend/  
agent/  
vision/  
gis/  
datasets/  
evaluation/  
models/  
storage/  
docs/  
tests/
```

Implement:

- Git repository
- Python environment
- Frontend environment
- Backend startup
- Environment configuration
- Logging
- Error handling
- Docker foundation
- Basic API health check

Exit condition:

The application starts successfully and every service can communicate.

No AI result is required yet.

PHASE 1 — REAL DATA INGESTION

Goal:

Build a reliable real-image ingestion system.

Implement:

- GeoTIFF reader
- TIFF reader
- Approved PNG/JPEG benchmark reader
- CRS validation
- Band detection
- Dimensions
- NoData detection
- Metadata extraction
- Input compatibility checks
- Image preview generation

Create a canonical metadata object:

```
{
  "source": "...",
  "sensor": "...",
  "modality": "...",
  "acquisition_date": "...",
  "crs": "...",
  "transform": "...",
  "width": 0,
  "height": 0,
  "bands": [],
  "nodata": null
}
```

Exit condition:

A real satellite image can be uploaded and completely validated.

PHASE 2 — SINGLE IMAGE INTELLIGENCE

Goal:

Complete the mandatory single-image baseline.

Implement:

VQA

Real image → VLM → answer

Scene description

Real image → VLM → description

Grounding

Real image + text → Grounding DINO → bounding boxes

Segmentation

Grounding box → SAM 2.1 → mask

Exit condition:

At least two complete single-image workflows work on real imagery.

PHASE 3 — REMOTE-SENSING ADAPTATION

Goal:

Convert at least one AI component from generic capability toward remote-sensing-specific capability.

Tasks:

- Prepare real training dataset
- Validate annotations
- Build training pipeline
- Run adaptation/fine-tuning
- Save checkpoint
- Record experiment configuration
- Evaluate adapted model
- Integrate adapted model into inference

No fabricated training metrics.

Every reported metric must come from an actual evaluation run.

Exit condition:

The project can demonstrate a real adapted model.

PHASE 4 — BI-TEMPORAL CHANGE ENGINE

Goal:

Implement real change analysis.

Pipeline:

```
T1
+
T2
↓
Compatibility check
↓
Registration check
↓
AOI extraction
↓
Quality masking
↓
Metric generation
↓
```

```
Temporal comparison
↓
Noise filtering
↓
Change mask
↓
Polygonization
↓
Area calculation
↓
Evidence
```

Implement:

- NDVI difference
- NDWI difference
- SAR change
- Thresholding
- Connected components
- Polygonization
- Area
- GeoJSON

Do not use SAM merely to rediscover raster change masks.

The architecture explicitly rejects that unnecessary approach.

Exit condition:

A real pair of images produces a real change map and measurable statistics.

PHASE 5 — CHANGE UNDERSTANDING / CHANGE VQA

Goal:

Move from merely detecting change to explaining it.

Input:

T1 + T2 + query

Example:

“What changed between these two dates?”

Output:

- Change description
- Change locations
- Evidence
- Map
- Confidence

Integrate CDVQA benchmark evaluation where applicable.

Exit condition:

System can answer real temporal questions rather than simply producing a raster difference.

PHASE 6 — OPTICAL + SAR CROSS-MODAL ANALYSIS

Goal:

Implement the mandatory cross-modal capability.

Input:

Optical image
+
SAR image
+
Natural-language query

Tasks:

1. Validate both images.
2. Verify geographic compatibility.
3. Extract corresponding AOI.
4. Process optical information.
5. Process SAR information.
6. Combine evidence.
7. Generate joint answer.
8. Display modality-specific evidence.

Example:

“Use the optical and SAR images together to identify built-up and water-covered regions.”

Exit condition:

A real optical/SAR pair generates a genuine multimodal analysis result.

PHASE 7 — AGENTIC INTEGRATION

Goal:

Connect all capabilities through one controlled decision engine.

Example:

User:

“Has the built-up area increased between the two dates?”

Agent determines:

```
Task = change analysis
Inputs = temporal pair
Object = built-up area
Measurement = increase/decrease
```

Then selects:

```
registration
+
change analysis
+
object/land-cover analysis
+
geometry measurement
+
evidence synthesis
```

Another query:

“Highlight the water body.”

Agent selects:

```
grounding
+
segmentation
+
polygonization
```

The user should NOT have to manually select the model.

Exit condition:

The same UI can intelligently route different queries to different specialist workflows.

PHASE 8 — EVIDENCE / AUDITABILITY

Every completed request must generate an evidence object.

Required structure:

```
{
  "request_id": "...",
  "query": "...",
  "intent": "...",
  "sources": [],
  "aoi": {},
  "quality": {},
  "metrics": {},
  "geometry": {},
  "model_outputs": {},
  "artifacts": {},
  "execution_trace": []
}
```

Evidence must include, where applicable:

- Source
- Date
- Sensor
- Modality
- CRS
- Transform
- Processing operations
- Thresholds
- Model/version
- Confidence
- GeoJSON
- Metrics
- Quality checks

This matches the architecture's evidence/provenance requirements.

PHASE 9 — FRONTEND FINALIZATION

Build the final analyst interface.

Required layout:

	SatQuery AI				

	Upload/Input	Natural Language Query			

	IMAGE VIEW	MAP / GEOJSON			

	Analysis Result				

	Evidence	Metrics	Confidence	Execution	

The frontend should show the difference between:

- Model interpretation
- Deterministic measurement
- Evidence
- Final answer

PHASE 10 — EVALUATION

Do not evaluate using arbitrary hand-picked examples only.

Create automated evaluation pipelines for:

VQA

- Answer correctness

Grounding

- Precision
- Recall
- Localization quality

Segmentation

- IoU

Change detection

- Change IoU
- Area error

Registration

- Alignment quality

System

- Routing accuracy
- Failure rate
- End-to-end latency

These are aligned with the evaluation structure in the architecture.

PHASE 11 — REAL FAILURE TESTING

Test genuine failures.

Examples:

Missing CRS

Expected:

“CRS missing; analysis cannot be safely geo-grounded.”

Poor optical coverage

System evaluates usable AOI coverage and may select SAR.

Registration failure

System must not claim precise change geometry.

Grounding failure

System reports candidate/low-confidence status.

Model unavailable

System reports legitimate degradation.

Oversized image

Use AOI/windowed processing.

These failure modes are already defined in the architecture.

PHASE 12 — DEPLOYMENT

Prepare:

- Docker
- Backend
- Frontend
- Model services
- PostgreSQL/PostGIS
- Object storage
- Configuration
- Logging
- Health checks

Architecture target:

```
frontend
api
orchestrator
raster-service
vision-service
postgres-postgis
minio
```

Optional model serving:

vLLM.

The architecture recommends vLLM as the first serving target for Qwen3-VL.

15. DATA FLOW RULE

Never allow the following:

```
User
↓
LLM
↓
Invent answer
```

Correct architecture:

```
User query
↓
Agent
↓
Tool/model selection
↓
Real data
↓
Real inference / GIS computation
↓
Structured result
↓
Evidence
↓
Language synthesis
↓
Answer
```

The final language model must consume the evidence rather than invent measurements.

16. AI VS GIS RESPONSIBILITY BOUNDARY

AI may:

- Understand natural language
- Interpret image content
- Identify semantic objects
- Answer VQA
- Generate descriptions
- Suggest task/tool selection
- Explain existing evidence

AI must NOT:

- Calculate geographic coordinates
- Calculate polygon area
- Perform CRS conversion as a textual guess
- Invent spectral indices
- Invent image dates
- Invent spatial measurements

GIS/raster engine must:

- Calculate indices
- Calculate coordinates
- Transform CRS
- Calculate geometry

- Calculate area
- Generate polygons
- Validate raster metadata

This boundary is mandatory.

17. DATA STORAGE

Object storage

Store:

- Original imagery
- Processed imagery
- Preview imagery
- GeoJSON
- Reports
- Model artifacts

PostgreSQL + PostGIS

Store:

- Metadata
- AOIs
- Job records
- Geometry
- Analysis results

Do not embed large raster files directly into the relational database.

This follows the architecture's storage design.

18. API CONTRACT

Required minimum endpoints:

```
POST /jobs
GET /jobs/{id}
WS /jobs/{id}/stream

POST /datasets/validate

GET /layers/{id}
```

```
POST /explain/{id}
```

These correspond to the architecture's proposed API surface.

19. NON-FUNCTIONAL REQUIREMENTS

The system must be:

- Reproducible
- Modular
- Observable
- Testable
- Deterministic where scientific calculations are involved
- Auditable
- Fault tolerant
- Dataset-aware
- CRS-aware
- Real-data driven

Every request should have a unique request ID.

Every model execution should be traceable.

Every spatial output should have provenance.

20. INTEGRATION RULE

Each person owns a subsystem, but nobody is allowed to create an isolated component that cannot integrate.

At the end of every phase:

```
Developer branch
  ↓
Unit tests
  ↓
Integration test
  ↓
Main branch
  ↓
System regression test
```

No component is considered complete merely because its individual code runs.

It is complete only when:

1. It processes real input.
 2. It generates real output.
 3. The output has proper schema.
 4. The output can be consumed by the next subsystem.
 5. Failure states are handled.
 6. The output can be displayed or evaluated.
-

21. DEFINITION OF DONE

A feature is DONE only when all of the following are true:

- ☐ Real input supported
 - ☐ Real processing performed
 - ☐ No synthetic data
 - ☐ No hardcoded result
 - ☐ No simulated output
 - ☐ Error handling implemented
 - ☐ Logging implemented
 - ☐ Output schema defined
 - ☐ Provenance recorded
 - ☐ Frontend integration complete
 - ☐ Evaluation test exists
 - ☐ Documentation exists
-

22. FINAL REQUIRED DEMONSTRATIONS

The finished project must demonstrate these four scenarios.

DEMO 1 — VQA

Input:

Real single satellite image.

Query:

“Describe the land-cover and major objects visible in this image.”

Result:

Real answer + evidence.

DEMO 2 — GROUNDING / SEGMENTATION

Input:

Real satellite image.

Query:

"Highlight the water body."

Result:

Grounding box + segmentation mask + GeoJSON + map.

DEMO 3 — TEMPORAL CHANGE

Input:

Real T1 and T2 images.

Query:

"What changed between these two dates, and where did the change occur?"

Result:

Real change analysis + change description + polygons + measurement.

DEMO 4 — OPTICAL + SAR

Input:

Real optical + SAR pair.

Query:

"Use the optical and SAR images together to identify built-up and water-covered regions."

Result:

Real cross-modal reasoning + visual evidence.

23. FLAGSHIP DEMO

Use the architecture's multi-step concept as the primary demonstration:

"Show me where flood water increased between these two dates, calculate the affected area, and tell me whether vegetation also decreased."

Expected workflow:

```
Natural-language query
↓
Agent planning
↓
Image validation
↓
Optical/SAR quality decision
↓
NDWI temporal analysis
↓
Change mask
↓
Polygonization
↓
Area calculation
↓
NDVI temporal analysis
↓
Evidence package
↓
Language explanation
↓
Interactive map
```

This demonstrates that the application is more than a VLM chatbot because several real analytical operations are being coordinated.

24. DATASET POLICY

Use official/public datasets only.

Primary datasets:

BigEarthNet.txt

Purpose: Remote-sensing adaptation/fine-tuning.

VRSBench

Purpose: Captioning, grounding and VQA evaluation.

RSVQA

Purpose: Remote-sensing VQA.

CDVQA

Purpose: Bi-temporal change VQA.

The SIH statement explicitly associates these datasets with adaptation and evaluation.

The ISRO/SAC dataset is an evaluation dataset whose annotations are not provided to teams. The system must therefore be designed to operate on that expected real data format without assuming access to its hidden labels.

25. IDE CODING BEHAVIOUR

The IDE agent must follow these rules while coding.

Before implementing a feature:

1. Identify which phase it belongs to.
2. Identify the owning component.
3. Identify the input/output contract.
4. Check whether the implementation requires real data.
5. Implement the smallest production-capable version.
6. Test using real available data.
7. Integrate it with the adjacent component.
8. Document what was implemented.

Do NOT:

- Create unnecessary abstractions
- Add random features
- Replace real ML with hardcoded rules
- Replace real data with generated data
- Create fake responses
- Hide failures
- Claim unsupported accuracy
- Pretend a component works without real execution

When something cannot run because the required model/dataset/hardware is unavailable, report the exact dependency and implement the correct interface rather than fabricating a successful result.

26. IMPLEMENTATION PRIORITY

Priority order:

```
P0 – Real data ingestion
P0 – Single-image VQA
P0 – Grounding/segmentation
P0 – Temporal change analysis
P0 – Optical + SAR analysis
P0 – Agentic routing
P0 – Evidence/provenance

P1 – Remote-sensing adaptation
P1 – Evaluation framework
P1 – Streaming execution trace
P1 – Failure handling
P1 – Report generation

P2 – Advanced optimization
P2 – Semantic retrieval
P2 – Advanced deployment optimization
P2 – Additional modalities/features
```

Do not implement P2 before the P0 system works end-to-end.

27. FINAL ARCHITECTURAL PRINCIPLE

The final product must be understood as five interconnected systems:

```
1. INTERFACE
Natural-language query + image upload

2. AGENT
Understands the task and chooses the workflow

3. AI
Provides semantic understanding

4. GEOSPATIAL ENGINE
Performs real scientific/spatial computation

5. EVIDENCE SYSTEM
Combines everything into a verifiable result
```

The core differentiator is:

The AI decides what to do; specialist models perform perception; deterministic geospatial processing performs scientific computation; the evidence layer proves how the answer was produced.

This is the implementation philosophy that the rest of the codebase must preserve.

28. FINAL ACCEPTANCE TEST

The project is considered complete only when all statements below are TRUE:

- [] User can upload real satellite imagery
- [] System validates imagery
- [] User can ask natural-language questions
- [] Agent identifies task automatically
- [] Single-image VQA works
- [] Additional single-image task works
- [] Bi-temporal analysis works
- [] Optical + SAR analysis works
- [] At least one component has genuine remote-sensing adaptation
- [] Grounding works where applicable
- [] Segmentation works where applicable
- [] Spatial geometry is genuinely calculated
- [] Change maps are genuinely calculated
- [] Results are generated from real inputs
- [] No synthetic data is used
- [] No simulated analysis is used
- [] No static demo results are used
- [] No hardcoded answer replaces inference
- [] Evidence is generated for every result
- [] Confidence is based on actual measurable signals
- [] Execution trace is visible
- [] Failures are transparently reported
- [] Benchmark evaluation is reproducible
- [] System is ready for unseen ISRO/SAC evaluation data

The system should always prefer **a truthful incomplete result over a fabricated complete result.**